



Лабораторная работа 2
"Контроль температуры"

Группа: Z3301

Студент: Вестфаль Мишель, Карасев Георгий, Ушмакова Дарья

Преподаватель: Денисов Константин Михайлович

1 Цель

Комбинируя интервалы включенного и выключенного состояния нагревателей (лампочек) с включением и выключением вентилятора, обеспечить требуемый температурный режим, который контролируется датчиком температуры.

2 Задачи

1. Реализовать функции управления состоянием нагревателей (ламп) и вентилятора
2. Реализовать обработку сигнала датчика температуры и вычисление температуры
3. Реализовать протокол взаимодействия микропроцессора и ПК с помощью интерфейса UART

3 Блок схема кода

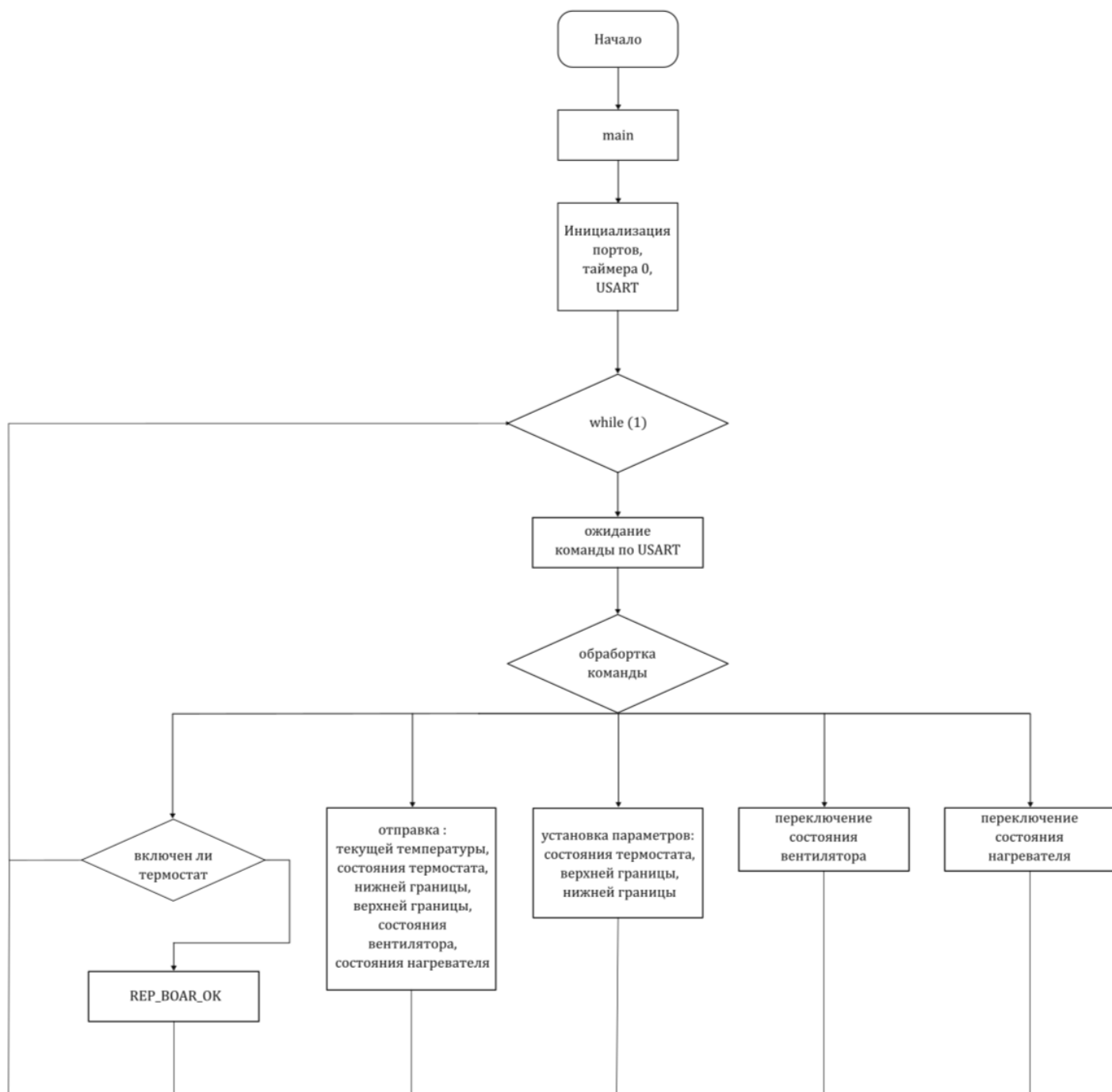


Рис. 1: Блок схема основной части кода



Рис. 2: Блок схема обработки прерываний

4 Пояснение к реализации

4.1 Общая структура программы

Программа для микроконтроллера GD32F303CC реализует систему контроля температуры на основе данных от датчика температуры, вентилятора и трёх нагревательных элементов (ламп). Управление устройствами осуществляется через порты ввода-вывода микроконтроллера, а обмен данными с компьютером производится по интерфейсу USART.

4.2 Инициализация периферии

Для работы термостата используются следующие выводы:

После инициализации на всех управляющих выводах устанавливается логическая единица, что соответствует выключенному состоянию нагревателей и вентилятора.

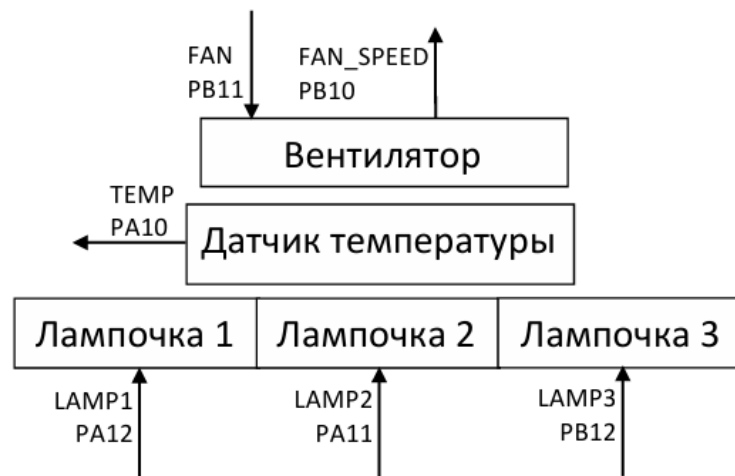


Рис. 3: Пины платы термостата

4.3 Управление нагревателями и вентилятором

При вызове функции `heater_on()` на выводы PA11, PA12 и PB12 подаётся логический ноль, вследствие чего открываются транзисторные ключи и включаются все три лампы.

Функция `heater_off()` устанавливает логическую единицу на соответствующих выводах и отключает нагреватели.

Управление вентилятором осуществляется аналогично функциями: `void fan_on(void)`, `void fan_off(void)`.

Поскольку схема управления вентилятором также активна по низкому уровню сигнала, включение вентилятора выполняется установкой логического нуля на выводе PB11.

4.4 Изменение температуры

Измерение температуры выполняется с использованием режима Input Capture таймера TIMER0. Сначала `TIMER_IC_POLARITY_RISING` фиксирует фронт, после прерывания `TIMER_IC_POLARITY_FALLING` фиксирует спад.

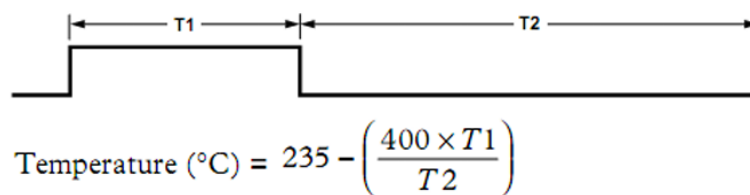


Рис. 4: Сигнал с датчика температуры

Таймер TIMER0 настроен таким образом, чтобы его частота счёта составляла 1 МГц: `timer_initpara.p` = 71.

При тактовой частоте 72 МГц это обеспечивает разрешение измерения 1 мкс.

Сигнал датчика подключён к каналу захвата TIMER0. В обработчике прерывания производится попеременное измерение времени высокого и низкого уровней сигнала. Для этого после каждого захвата изменяется полярность входного канала между фронтом и спадом.

После получения обоих временных интервалов вычисляется температура: `temperature = 235.0 - (400.0 * diff_up) / diff_down`. Полученное значение сохраняется в глобальной переменной `temperature`.

4.5 Реализация термостата

Автоматическое поддержание температуры реализовано функцией `update_thermostat()`.

Пользователь задаёт верхний и нижний температурные пороги: `limit_hi_temp`, `limit_low_temp`.

Алгоритм работы следующий:

если температура превышает верхний предел, включается вентилятор и выключаются нагреватели;

если температура опускается ниже нижнего предела, включаются нагреватели и выключается вентилятор.

4.6 Обмен данными по USART

Для взаимодействия с персональным компьютером используется интерфейс USART1 с параметрами: скорость — 9600 бит/с, 8 бит данных, без контроля четности и с 1 стоп-битом.

Передача и приём данных реализованы функциями: `usart1_transmit()`, `usart1_receive()`.

4.7 Реализация протокола обмена

Обмен между компьютером и микроконтроллером построен на основе набора команд.

Проверка связи - команда 0x11 – ответ 0x12

Запрос состояния системы - команда 0x14 – в ответ передаются текущая температура, состояние термостата, верхний и нижний температурный порог, состояние вентилятора и нагревателей.

Установка режима работы - команда 0x15 – прием 3 байт (состояние термостата, верхняя и нижняя граница температуры). Полученные значения сохраняются в переменных: `thermostat_is_on`, `limit_hi_temp`, `limit_low_temp`.

Управление вентилятором - команда 0x16 – инвертирует текущее состояние.

Управление нагревателем - команда 0x17 – инвертирует текущее состояние.

4.8 Система синхронизации

Для работы периферии необходимо настроить систему тактирования. В `main()` есть `SystemInit()` для формирования системной частоты $СК_{AHB} = 120 \text{ МГц}$, затем обе шины делятся на 4: $APB1 = 120 / 4 = 30 \text{ МГц}$.

5 Код

```
1 #include "gd32f30x.h"
2 #include "systick.h"
3
4 void systick_config(void);
5 void PORT_INIT(void);
6 void SysTick_Handler(void);
7 void TIMER_INIT(void);
8 void Timer0IC_Init(void);
9 void Interrupt_Timer0IC_Init(void);
10
11
12 /* ----- Константы ----- */
13 typedef uint32_t BOOL;
14 #define TRUE 0x01
```

```

15 #define FALSE 0x00
16
17 /* Протокольные коды (1 байт) */
18 static const uint8_t REQ_INIT = 0x11;
19 static const uint8_t REP_BOAR_OK = 0x12;
20 static const uint8_t REQUEST_MODE = 0x14;
21 static const uint8_t SET_MODE = 0x15;
22 static const uint8_t TOGGLE_FAN = 0x16;
23 static const uint8_t TOGGLE_HEATER = 0x17;
24
25 /* ----- Системные переменные ----- */
26 volatile float temperature = 30.0f;
27 volatile BOOL fan_is_on = FALSE;
28 volatile BOOL heater_is_on = FALSE;
29 volatile BOOL thermostat_is_on = TRUE;
30 volatile uint8_t limit_hi_temp = 30; /* произвольные стартовые значения */
31 volatile uint8_t limit_low_temp = 20;
32
33 volatile static timer_ic_parameter_struct timer0_ch2_paraminit = {
34     TIMER_IC_POLARITY_RISING,
35     TIMER_IC_SELECTION_DIRECTTI,
36     TIMER_IC_PSC_DIV1,
37     1
38 };
39
40 void PORT_INIT(void)
41 {
42     rcu_periph_clock_enable(RCU_GPIOA); //синхронизировать //Enable clock for
43     GPIOA
44     rcu_periph_clock_enable(RCU_GPIOB); //синхронизировать //Enable clock for
45     GPIOB
46
47     gpio_init(GPIOA, GPIO_MODE_IN_FLOATING, GPIO_OSPEED_10MHZ, GPIO_PIN_10);
48     //temperature
49     gpio_init(GPIOB, GPIO_MODE_OUT_OD, GPIO_OSPEED_10MHZ, GPIO_PIN_11); //FAN
50     gpio_init(GPIOB, GPIO_MODE_OUT_OD, GPIO_OSPEED_10MHZ, GPIO_PIN_10);
51     //FAN_SPEED
52     gpio_init(GPIOB, GPIO_MODE_OUT_OD, GPIO_OSPEED_10MHZ, GPIO_PIN_12);
53     //LAMP3
54     gpio_init(GPIOA, GPIO_MODE_OUT_OD, GPIO_OSPEED_10MHZ, GPIO_PIN_11);
55     //LAMP2
56     gpio_init(GPIOA, GPIO_MODE_OUT_OD, GPIO_OSPEED_10MHZ, GPIO_PIN_12);
57     //LAMP1
58
59     gpio_bit_set(GPIOA, GPIO_PIN_12);
60     gpio_bit_set(GPIOB, GPIO_PIN_12);
61     gpio_bit_set(GPIOA, GPIO_PIN_11);
62     gpio_bit_set(GPIOB, GPIO_PIN_11);
63 }
64
65 void TIMERO_INIT(void)
66 {
67     timer_parameter_struct timer_initpara;

```

```

61
62 rcu_periph_clock_enable(RCU_TIMER0); //sinchronise //Enable clock for
    GPIOA
63
64 timer_initpara.prescaler = 71; //F_clk / (prescaler + 1) => 72 MHz / (71
    + 1) = 1 MHz
65 timer_initpara.alignedmode = TIMER_COUNTER_EDGE;
66 timer_initpara.counterdirection = TIMER_COUNTER_UP;
67 timer_initpara.period = 0xFFFF;
68 timer_initpara.clockdivision = TIMER_CKDIV_DIV1;
69 timer_initpara.repetitioncounter = 0;
70
71 timer_auto_reload_shadow_enable(TIMER0); // if we change ARR -> new ARR
    start after end last period of work
72 timer_init(TIMER0, &timer_initpara);
73 }
74
75 void Timer0IC_Init(void){
76     rcu_periph_clock_enable(RCU_TIMER0); //
77     timer_input_capture_config(TIMER0, TIMER_CH_2, &timer0_ch2_paraminit);
78     set_timer0IC_icpolarity(TIMER_IC_POLARITY_FALLING);
79 }
80
81 void USART1_init(void)
82 {
83     rcu_periph_clock_enable(RCU_GPIOA);
84     rcu_periph_clock_enable(RCU_USART1);
85
86     gpio_init(GPIOA, GPIO_MODE_AF_PP, GPIO_OSPEED_50MHZ, GPIO_PIN_2);
87     gpio_init(GPIOA, GPIO_MODE_IN_FLOATING, GPIO_OSPEED_50MHZ, GPIO_PIN_3);
88
89     // USART configure
90     usart_deinit(USART1);
91     usart_baudrate_set(USART1, 9600U);
92     usart_parity_config(USART1, USART_PM_NONE);
93     usart_word_length_set(USART1, USART_WL_8BIT);
94     usart_stop_bit_set(USART1, USART_STB_1BIT);
95     usart_receive_config(USART1, USART_RECEIVE_ENABLE);
96     usart_transmit_config(USART1, USART_TRANSMIT_ENABLE);
97 }
98
99 void set_timer0IC_icpolarity(uint16_t polarity)
100 {
101     timer0_ch2_paraminit.icpolarity = polarity;
102     timer_input_capture_config(TIMER0, TIMER_CH_2, &timer0_ch2_paraminit);
103 }
104
105 volatile uint32_t diff_up = 0;
106 volatile uint32_t diff_down = 0;
107 volatile uint32_t time_interrupt;
108
109 volatile BOOL is_exist_up_data = FALSE;
110 volatile BOOL is_exist_down_data = FALSE;

```

```

111 volatile BOOL is_rising_interrupt = FALSE;
112
113
114 void TIMERO_Channel_IRQHandler(void)
115 {
116     time_interrupt = (timer_channel_capture_value_register_read(TIMERO,
117         TIMER_CH_2));
118
119     if(is_rising_interrupt)
120     {
121         diff_down = time_interrupt;
122         set_timerOIC_icpolarity(TIMER_IC_POLARITY_FALLING);
123         timer_counter_value_config(TIMERO, 0);
124
125         is_rising_interrupt = FALSE;
126         is_exist_down_data = TRUE;
127     } else {
128         diff_up = time_interrupt;
129         set_timerOIC_icpolarity(TIMER_IC_POLARITY_RISING);
130         timer_counter_value_config(TIMERO, 0);
131
132         is_rising_interrupt = TRUE;
133         is_exist_up_data = TRUE;
134     }
135
136     if (is_exist_up_data && is_exist_down_data)
137     {
138         if (diff_down != 0) {
139             temperature = 235.0 - (400.0 * (double)diff_up)/ (double)diff_down;
140             update_thermostat();
141         }
142         is_exist_up_data = FALSE;
143         is_exist_down_data = FALSE;
144     }
145
146     timer_interrupt_flag_clear(TIMERO, TIMER_INT_FLAG_CH2);
147 }
148
149 void update_thermostat() {
150     if (temperature > limit_hi_temp) {
151         fan_on();
152         heater_off();
153     } else if (temperature < limit_low_temp) {
154         fan_off();
155         heater_on();
156     }
157 }
158
159 void usart1_transmit(uint16_t symbol)
160 {
161     while (RESET == usart_flag_get(USART1, USART_FLAG_TBE)){ // while
162         buffer not empty
163         usart_data_transmit(USART1, symbol);

```

```

162 }
163
164 uint16_t usart1_receive(void)
165 {
166     while (RESET == usart_flag_get(USART1, USART_FLAG_RBNE)){ } // while in
        buffer exist not read data
167     return usart_data_receive(USART1);
168 }
169
170 void heater_on(void)
171 {
172     gpio_bit_reset(GPIOA, GPIO_PIN_12);
173     gpio_bit_reset(GPIOB, GPIO_PIN_12);
174     gpio_bit_reset(GPIOA, GPIO_PIN_11);
175     heater_is_on = TRUE;
176 }
177
178 void heater_off(void)
179 {
180     gpio_bit_set(GPIOA, GPIO_PIN_12);
181     gpio_bit_set(GPIOB, GPIO_PIN_12);
182     gpio_bit_set(GPIOA, GPIO_PIN_11);
183     heater_is_on = FALSE;
184 }
185
186 void fan_on(void)
187 {
188     gpio_bit_reset(GPIOB, GPIO_PIN_11);
189     fan_is_on = TRUE;
190 }
191
192 void fan_off(void)
193 {
194     gpio_bit_set(GPIOB, GPIO_PIN_11);
195     fan_is_on = FALSE;
196 }
197
198 int main(void)
199 {
200     // System Configuration AHB Clock 120 MHz
201     SystemInit();
202
203     // APB1, APB2 Clock Setup
204     rcu_apb1_clock_config(RCU_APB1_CKAHB_DIV4); // APB1 Clock 120MHz/4 =
30 MHz
205     rcu_apb2_clock_config(RCU_APB2_CKAHB_DIV4); // APB2 Clock 120MHz/4 =
30 MHz
206
207     // NVIC Configuration: 4 bits for priority, 0 bits for subpriority
208     nvic_priority_group_set(NVIC_PRIGROUP_PRE4_SUB0);
209
210     systick_config();
211

```

```

212 PORT_INIT();
213
214 TIMER0_INIT();
215 timer_enable(TIMER0);
216
217 Timer0IC_Init();
218 timer_interrupt_enable(TIMER0, TIMER_INT_CH2);
219
220 USART1_init();
221 usart_enable(USART1);
222
223 //set priority on timperature controller
224 nvic_irq_enable(TIMER0_Channel_IRQn, 1, 0); //Timer 0 main_prior = 1,
subprior = 0
225
226 while (1)
227 {
228     uint16_t input = usart1_receive();
229     switch (input) {
230         case REQ_INIT:
231             if (thermostat_is_on == TRUE) {
232                 usart1_transmit(REP_BOAR_OK);
233             }
234             break;
235         case REQUEST_MODE:
236             /*
237              * Последовательно отправляем
238              * температуру
239              * включен ли режим термостата (0x00 не включен, 0x01 включен)
240              */
241             uint8_t mode_buf[6];
242
243             mode_buf[0] = (uint8_t)temperature; /*
Текущая температура целая( часть) */
244             mode_buf[1] = thermostat_is_on ? 0x01 : 0x00; /*
Термостат включён? */
245             mode_buf[2] = limit_hi_temp; /*
Верхняя граница */
246             mode_buf[3] = limit_low_temp; /*
Нижняя граница */
247             mode_buf[4] = fan_is_on ? 0x01 : 0x00; /*
Вентилятор */
248             mode_buf[5] = heater_is_on ? 0x01 : 0x00; /*
Нагреватель */
249
250             for (uint8_t i = 0; i < sizeof(mode_buf); ++i)
251                 usart1_transmit(mode_buf[i]);
252             break;
253         case SET_MODE:
254             /*
255              * Принимаем байт с номером устанавливаемого режима
256              * байт режима терсмостата (0x00 выключить термостат, включить
термостат 0x01)

```

```

257         байт задание верхней границы температуры
258         байт задание нижней границы
259         */
260         uint8_t mode_flag = uart1_receive(); /* 0x00 -
отключить, 0x01 - включить */
261         uint8_t hi_limit = uart1_receive(); /* верхняя
граница */
262         uint8_t lo_limit = uart1_receive(); /* нижняя
граница */
263
264         thermostat_is_on = (mode_flag == 0x01) ? TRUE : FALSE;
265         limit_hi = hi_limit;
266         limit_lo = lo_limit;
267
268         break;
269     case TOGGLE_FAN:
270         /*
271         Если вентилятор был включен, то он должен отключиться, если не
работал, то должен включиться
272         */
273         if (fan_is_on == TRUE)
274             fan_off();
275         else
276             fan_on();
277         break;
278     case TOGGLE_HEATER:
279         /*
280         Если нагреватель был включен, то он должен отключиться, если не
работал, то должен включиться
281         */
282         if (heater_is_on == TRUE)
283             heater_off();
284         else
285             heater_on();
286         break;
287     default:
288         break;
289 }
290 }
291 }
292
293 void SysTick_Handler(void)
294 {
295     delay_decrement();
296 }

```

Листинг 1: main.c